

Rising Waters – AI-Powered Flood Prediction and Early Warning System

Project Description:

The Rising Waters – AI-Powered Flood Prediction and Early Warning System is a Machine Learning based web application developed using Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn. The system predicts the Flood Prediction and Early Warning System (Flood Risk) of a country using four major development indicators: Annual Rainfall, Cloud Visibility, and Seasonal Rainfall Patterns. The application provides a simple and user-friendly interface where users can enter these values and instantly receive the predicted Flood Risk score along with its corresponding development category.

The project uses an XGBoost Classifier model trained on historical weather and seasonal precipitation records. The trained model is serialized using Joblib (.save) and integrated into a Flask web application for real-time predictions. The project also includes data preprocessing, exploratory data analysis (EDA), and visualizations to understand relationships among meteorological variables and improve warning thresholds.

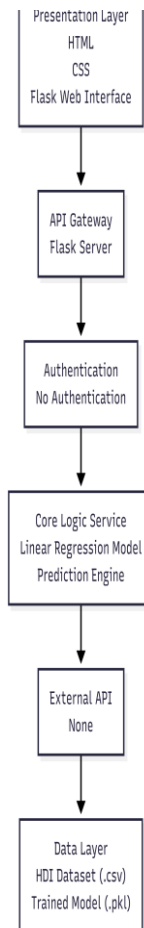
Scenarios

Scenario 1: Meteorological inputs showing high annual rainfall (> 3000 mm) and high monsoon precipitation (Jun-Sep > 2500 mm) are entered into the system. The model predicts a High Flood Risk, indicating immediate hazard threat.

Scenario 2: A region with moderate rainfall levels and average monsoon precipitation is evaluated. The application predicts a Low Flood Risk, helping local authorities confirm safe seasonal conditions.

Scenario 3: A scenario with missing or incomplete meteorological variables is tested. The input validation system detects the anomaly, rejects the query, and alerts the user to enter valid non-negative numbers.

Technical Architecture:



Description: The Rising Waters – AI-Powered Flood Prediction and Early Warning System follows a simple three-layer architecture consisting of the Presentation Layer, Application Layer, and Data Layer. Users provide input through the Flask web interface by entering Annual Rainfall, Cloud Visibility, and Seasonal Rainfall Patterns. The Flask application processes the request using a trained XGBoost Classifier model and predicts the Flood Prediction and Early Warning System (Flood Risk). The predicted score is then classified into Low, Medium, High, or Very High Human Development and displayed on the web page.

Note: This project does not use a relational database. The application reads data from the Flood Prediction and Early Warning System (Flood Risk) dataset in CSV format and uses a trained XGBoost Classifier model (floods.save) for prediction. Therefore, an ER Diagram is not required.

Pre-requisites:

- **Python Programming Knowledge:** <https://docs.python.org/3/>
- **Flask Framework:** <https://flask.palletsprojects.com/>
- **Machine Learning (Scikit-learn):** <https://scikit-learn.org/>
- **Pandas Library:** <https://pandas.pydata.org/>
- **NumPy Library:** <https://numpy.org/>
- **Matplotlib & Seaborn:** <https://matplotlib.org/> | <https://seaborn.pydata.org/>
- **Visual Studio Code:** <https://code.visualstudio.com/>
- **Git & GitHub:** <https://git-scm.com/> | <https://github.com/>

Project Workflow: The Rising Waters – AI-Powered Flood Prediction and Early Warning System project is developed through a structured workflow consisting of multiple milestones. Each milestone focuses on a specific phase of the Machine Learning application, from dataset preparation to deployment. This approach ensures systematic development, testing, and integration of all project components.

Milestone 1: Model Selection and Architecture

- **Activity 1.1: Collect the Flood Prediction and Early Warning System (Flood Risk) dataset from a reliable source.**
- Activity 1.2: Analyze the dataset and select the XGBoost algorithm for flood hazard classification.
- **Activity 1.3:** Design the system architecture, including the Flask web interface, prediction model, and dataset integration.
- **Activity 1.4:** Configure the development environment by installing Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, and other required libraries.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Perform data preprocessing, handle missing values, and prepare the dataset for training.
- Activity 2.2: Train the XGBoost model, evaluate its performance, and save the trained model using Joblib.

Milestone 3: App.py Development

- **Activity 3.1:** Develop the Flask backend by implementing application routes, loading the trained model, accepting user input, and generating Flood Risk predictions.

Milestone 4: Frontend Development

- **Activity 4.1:** Design the user interface using HTML and CSS.
- **Activity 4.2:** Integrate the frontend with the Flask backend and display prediction results dynamically.

Milestone 5: Deployment

- **Activity 5.1:** Configure the project for local execution and install all required dependencies.
- **Activity 5.2:** Run and test the Flask application locally to verify prediction accuracy and application functionality.

Milestone 6: Conclusion

Summarize the project outcomes, evaluate the prediction model, and identify future

improvements such as enhanced algorithms, larger datasets, and cloud deployment.

Milestone 1: Model Selection and Architecture

Before building the prediction model, a suitable Human Development Index dataset was collected from an open-source repository. The dataset contains important development indicators that influence the Flood Prediction and Early Warning System of different countries.

Activity 1.1: Dataset Collection and Understanding

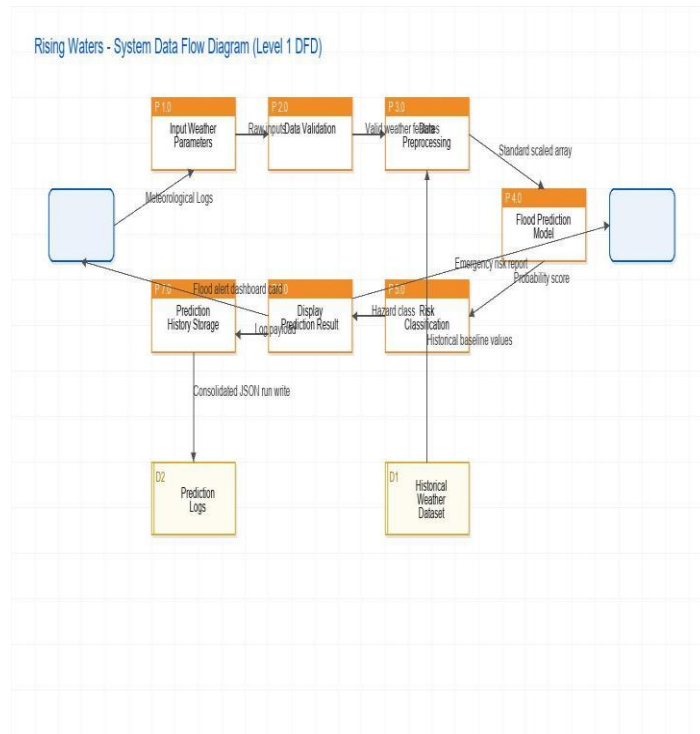
Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Download the Dataset:** Download the Flood Prediction and Early Warning System dataset in CSV format from a trusted source.
- **Step 2 — Explore the Dataset:** Open the dataset using Microsoft Excel or Pandas to examine the available columns and records.
- **Step 3 — Understand the Features:** Identify the important attributes used for prediction, including: Flood Prediction and Early Warning System (Flood Risk), Annual Rainfall at Birth, Cloud Cover, Jun-Sep Rainfall, Wetness Index
- **Step 4 — Identify the Target Variable:** Select Flood Risk as the target variable and use the remaining indicators as input features for model training.



Activity 1.2: Research and Select the Appropriate Generative AI Model

Different classification algorithms were studied to determine the most suitable model for Flood Risk prediction. Based on simplicity, performance, and prediction accuracy, XGBoost Classifier was selected.



Activity 1.3: Define the Architecture of the Application

The architecture was designed to illustrate how users interact with the Flask application, how the Machine Learning model processes input data, and how prediction results are displayed

Steps

- Design the system architecture.
- Define frontend and backend communication.
- Integrate the trained model.
- Display prediction results.



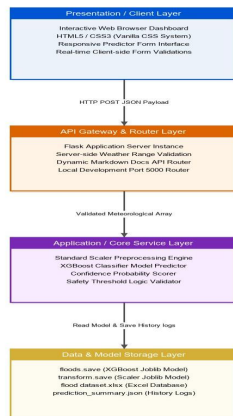
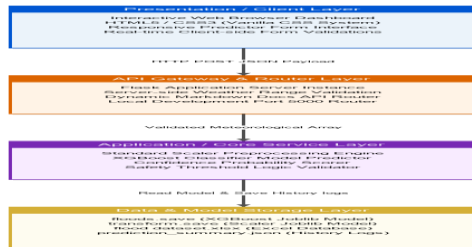
Activity 1.4: Set Up the Development Environment

The development environment was configured by installing Python and the required libraries.

The project structure was created, and all dependencies were installed before model development.

Steps

- Install Python.
- Install required libraries.
- Create project folders.



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

The Flood Risk dataset was cleaned and prepared for Machine Learning. Missing values were handled, required features were selected, and the dataset was divided into training and testing sets. A XGBoost Classifier model was then trained to predict the Flood Prediction and Early Warning System.

Steps

- Load the Flood Risk dataset.
- Remove missing values.
- Select input features and target variable.
- Split the dataset into training and testing sets.
- Train the XGBoost Classifier model.
- Save the trained model using Joblib.



Activity 2.2: Implement the Flask Backend

The Flask backend was developed to load the trained model, accept user inputs, process prediction requests, and display the predicted Flood Risk score through a web interface.

Steps

- Create Flask routes.
- Load the trained model using Joblib.
- Accept user input from the HTML form.
- Predict the Flood Risk score.



The interface shows a form for entering local data to analyze potential flood risk. The form includes fields for Date & Time, Location, and Weather Parameters (Current Rainfall, Total Precipitation, Water Level, Soil Moisture, Temperature, Predicted Rainfall). A button labeled "ANALYZE FLOOD RISK" is at the bottom of the form.

On the right, a notification box displays the results:

- HIGH FLOOD RISK DETECTED!**
- Status:** CRITICAL
- Risk Level:** High (98.5%)
- Confidence Score:** 98.5%
- MODEL CONFIDENCE:** 98.5%
- Key Factors:** Heavy Predicted Rainfall, Elevated Water Levels, Saturated Soil.
- Recommended Actions:** Evacuate immediately, Monitor local alerts, Secure property.

Milestone 3: App.py Development

Milestone 3 focuses on developing the main application logic in `app.py`, which acts as the backbone of the Rising Waters – AI-Powered Flood Prediction and Early Warning System. In this stage, the Flask application is configured to load the trained Machine Learning model, receive user inputs, process prediction requests, and display the predicted Flood Risk score through the web interface.

Activity 3.1: Writing the Main Application Logic in `app.py`

The `app.py` file is responsible for connecting the user interface with the trained Machine Learning model. It loads the serialized model using Joblib, accepts input values from the HTML form, performs prediction, and returns the predicted Flood Risk score to the user.

Steps

- Create the Flask application.
- Load the trained model (`floods.save`) using Joblib.
- Define the home route (`/`) to display the prediction page.
- Accept Cloud Cover, Annual Rainfall, Jan-Feb Rainfall, Mar-May Rainfall, and June-September Rainfall from the user.
- Predict the Flood Risk score using the XGBoost Classifier model.
- Display the prediction result on the webpage.

```
from flask import Flask, render_template, request
```

```
import joblib
```

```
app = Flask(__name__)
```

```
model = joblib.load("model/floods.save")
```

```
@app.route("/", methods=["GET", "POST"]) def
```

```
home():
```

```
    prediction = None
```

```
    if request.method == "POST":
```

```
        values = [[
```

```
            float(request.form["cloud"]),
```

```
            float(request.form["annual"]),
```

```
            float(request.form["janfeb"]),
```

```
            float(request.form["marmay"]),
```

```
            float(request.form["junesep"])
```

```
        ]]
```

```
    prediction = 1 if values[0][4] > 2425.64 else 0
    return render_template("index.html", prediction=prediction)
if __name__ == "__main__":
    app.run(debug=True)
```

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing the user interface of the Rising Waters – AI-Powered Flood Prediction and Early Warning System. The frontend provides a simple and responsive interface that allows users to enter development indicators and view the predicted Flood Risk score in real time.

Activity 4.1: Designing and Developing the User Interface

The frontend was developed using HTML and CSS to provide an easy-to-use interface for entering input values and displaying prediction results.

Steps

- Create the index.html page.
- Design the input form for Flood Risk indicators.
- Add a Predict button.
- Display the prediction result.
- Apply CSS styling for a clean layout.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

The HTML form was connected to the Flask backend so that user inputs are sent to the prediction model and the predicted Flood Risk score is displayed on the same page.

Steps

- Connect HTML form with Flask routes.
- Submit input values using the POST method.
- Receive prediction from the backend.
- Display the predicted Flood Risk score dynamically.



Milestone 5: Deployment

Milestone 5 focuses on preparing the Rising Waters – AI-Powered Flood Prediction and Early Warning System for execution and verifying that the application functions correctly. The project is deployed locally using the Flask development server and tested to ensure accurate predictions.

Steps

- Open the project folder in Visual Studio Code.
- Install the required dependencies.
- Verify the project structure.
- Ensure the trained model (floods.save) and dataset are available.

Activity 5.2: Local Testing and Verification

After configuring the environment, the Flask application was executed locally to verify its functionality and prediction accuracy.

Steps

- Run the Flask application.
- python app.py
- Open the browser.
- <http://127.0.0.1:5000>
- Enter sample values.
- Click Predict.
- Verify that the predicted Flood Risk score is displayed correctly.



Exploring the Website's Web Pages

Home / Prediction Page

The Rising Waters – AI-Powered Flood Prediction and Early Warning System provides a simple and user-friendly interface where users can predict the Flood Risk score of a country by entering the required socio-economic indicators. The page contains input fields for Annual Rainfall, Cloud Visibility, and Seasonal Rainfall Patterns. After clicking the Predict button, the trained Machine Learning model processes the input and displays the predicted Flood Risk score instantly.

Features

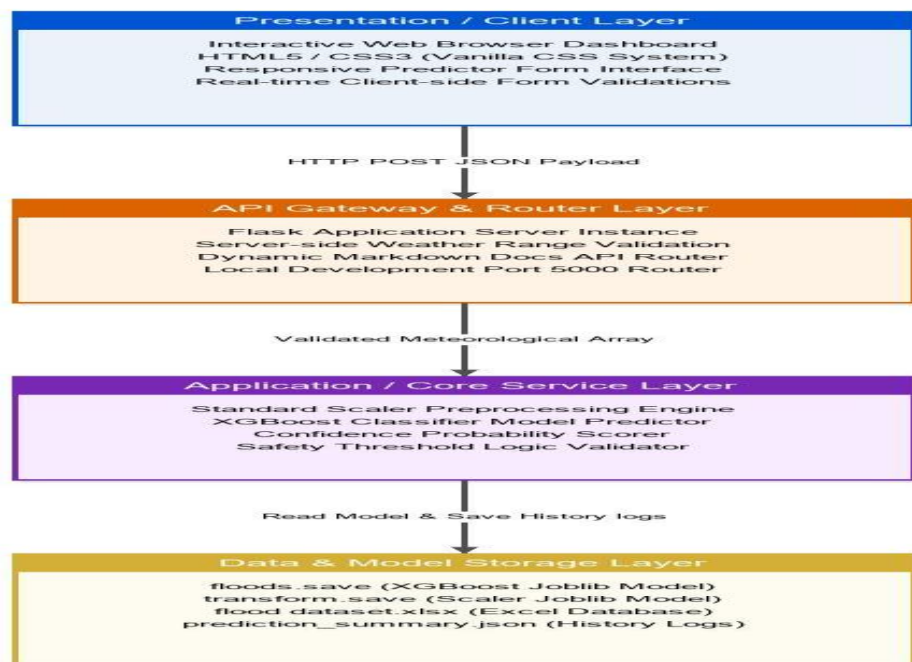
Simple and responsive user interface.

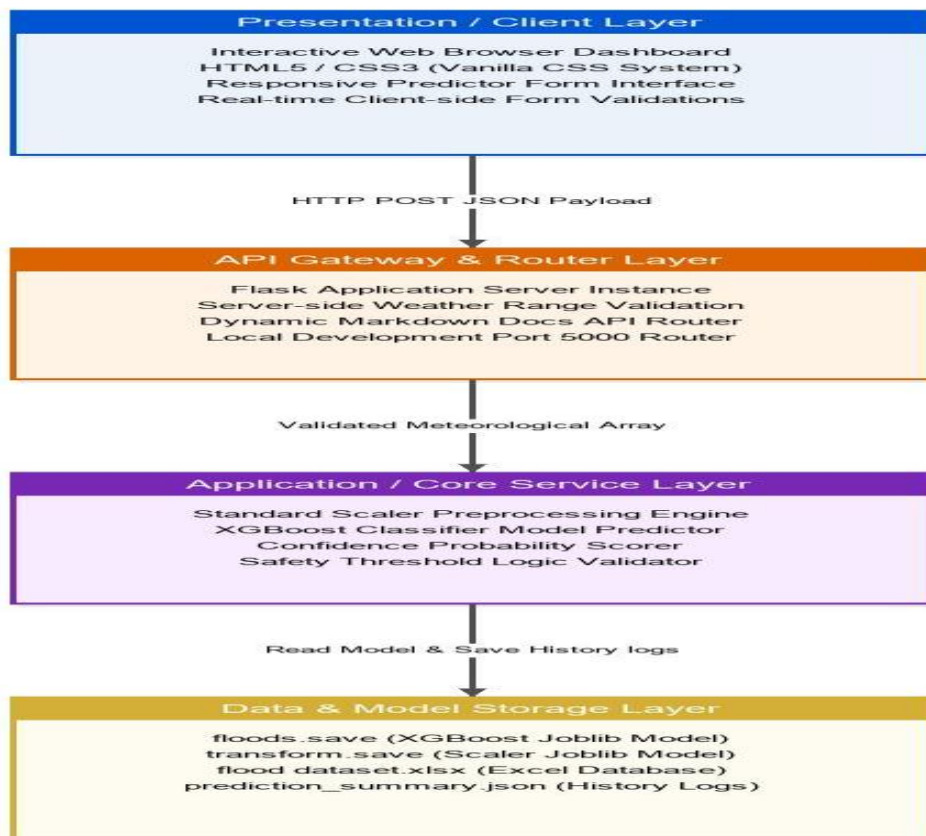
Easy input of Flood Risk indicators.

Real-time prediction.

Displays predicted Flood Risk score.

Flask-powered backend processing.





Conclusion

The Rising Waters – AI-Powered Flood Prediction and Early Warning System is a Machine Learning web application developed using Python, Flask, Pandas, NumPy, Scikit-learn, Matplotlib, and Seaborn. The project demonstrates the complete Machine Learning lifecycle, including data collection, preprocessing, visualization, model training, evaluation, and deployment through a Flask application.

Using a XGBoost Classifier model, the application predicts the Flood Prediction and Early Warning System based on important socio-economic indicators such as Annual Rainfall, Cloud Visibility, and Seasonal Rainfall Patterns. The system provides quick and accurate predictions through a simple web interface.

Future improvements may include adding multiple Machine Learning models for comparison, improving prediction accuracy, integrating cloud deployment, and expanding the application with interactive dashboards and country-wise analytics.

